COMPONENTE DE BORDE · TEMA 01 · UTN FRC

API Gateway y Service Discovery

Cómo funciona la única puerta de entrada del sistema, y cómo los microservicios se encuentran entre sí

Pipeline de filtros · Ruteo · Eureka · Resiliencia · Contratos

1 Dos conceptos previos

Qué problema resuelve cada componente antes de entrar en el diseño

API GATEWAY

Intermediario entre clientes y microservicios. Recibe las peticiones y centraliza autenticación, ruteo, límites, trazabilidad y políticas técnicas antes de reenviarlas.

Evita que el cliente conozca cada dirección y que todos los microservicios repitan las mismas tareas transversales.

SERVICE DISCOVERY

Registro donde cada microservicio publica su nombre lógico, ubicación y estado. El Gateway consulta ese registro para resolver instancias sin guardar direcciones fijas.

Permite que un contenedor cambie de host o IP sin modificar las rutas externas.

Una analogía

El Gateway es la recepción: única entrada, controla credenciales y dirige. Eureka es el directorio interno que informa dónde está disponible cada oficina.

En este proyecto: Spring Cloud Gateway WebFlux como gateway y Netflix Eureka como service discovery.

2 La puerta única también se impone en la red

El Gateway transporta requests; Eureka solo informa ubicaciones

1

CLIENTES

Web, móvil e integraciones externas solo conocen la URL pública del Gateway.

→

2

GATEWAY

Único puerto publicado. Valida credenciales, aplica políticas técnicas y reenvía.

→

3

MICROSERVICIOS

Puertos privados. Reciben tráfico del Gateway y autorizan reglas de negocio.

→

4

EUREKA

Plano de control: registra instancias. No transporta requests de negocio.

Regla de infraestructura

```
Internet → API Gateway → microservicio

micro A ⟶ API Gateway ⟶ micro B
BLOQUEADO: Internet → micro · micro A → micro B directo

Docker: red privada sin publicar puertos · Kubernetes: ClusterIP + NetworkPolicy
```

3

¿Qué hace realmente el API Gateway?

Infraestructura con autorización técnica, pero sin decisiones de dominio

Sí hace

Puerta única	Es el único componente expuesto; la red bloquea accesos directos.
Autenticar	Valida firma, issuer, audience, algoritmo y vigencia del JWT.
Rutear dinámico	Expone servicios admitidos y resuelve instancias mediante Eureka.
Propagar identidad	Reemplaza headers entrantes y propaga usuario o servicio verificado.
Proteger rutas internas	Iexige token y rol de servicio válido.
Proteger (resiliencia)	Aplica timeouts, límites, retry idempotente y circuit breaker.
Observar	Propaga trazas y registra métricas y logs sin secretos.

NO hace

Autorizar negocio	Qué puede hacer cada rol lo decide el microservicio.
Resolver ownership	¿Es su curso o desafío? Lo decide el dueño del dominio.
Conocer dominios	No sabe qué es un curso, un legajo ni un desafío.
Emitir tokens	Eso pertenece a users-service; el Gateway solo valida.
Guardar permisos de dominio	No mantiene una base de negocio ni un mapa funcional de roles.
Guardar sesión	La validación es stateless: verifica el token, no una sesión local.

Si una decisión necesita datos del dominio, pertenece al micro; si protege la frontera técnica, puede pertenecer al Gateway.

4

Ruteo estático vs. dinámico gobernado

Eureka resuelve instancias; el Gateway decide qué APIs se exponen

ESTÁTICO · cada ruta declarada

Catálogo central por servicio

A FAVOR

+ Cada API externa está aprobada explícitamente
+ Políticas y paths pueden variar por servicio

EN CONTRA

− Alta manual de cada servicio nuevo
− El catálogo central crece con el proyecto
− Puede convertirse en cuello de botella del equipo Gateway
− Repite configuración si todos siguen la misma convención

DINÁMICO · nuestra elección

El Gateway crea rutas desde servicios admitidos

A FAVOR

+ Onboarding mediante una convención única
+ Eureka aporta las instancias disponibles
+ Filtros genéricos se aplican a todos los servicios
+ No hay una ruta escrita por cada microservicio

RIESGOS A MITIGAR

− Exposición accidental → allowlist o include-expression
− API acoplada al serviceId → naming versionado y pruebas

Decisión: discovery dinámico gobernado. Eureka informa dónde está cada instancia; el Gateway controla qué servicios se exponen y con qué políticas.

5 Estándar de rutas y contratos

Discovery locator personalizado: {nombre}-service → /api/{nombre}/**

/api/{nombre}/public/**

Pueden ser públicas, autenticadas o internas según la regla del microservicio(usar anotación @RolesAllowed).

`/api/users/public/auth/login`

/api/{nombre}/**

Pueden ser públicas, autenticadas o internas según la regla del microservicio(usar anotación @RolesAllowed).

`/api/users/me`

El {nombre} en la ruta se deriva del serviceId

users-service

→ /api/users

mailing-service

→ /api/mailing

api-gateway

→ excluido del locator

*El locator por defecto usa /serviceId/** y RewritePath. Nuestra regla debe declarar el predicate y el path final enviado al backend.*

Dos excepciones controladas

`/.well-known/jwks.json` → endpoint del emisor. `/api/users/auth/token` → exige Client Credentials o mTLS; nunca queda abierto.

6 Un contrato de nombres aplicado en tres lugares

La convención ayuda a ubicar el código; el serviceId es la fuente del ruteo

Nombre del repositorio	`tpi-{nombre}`	→	**`tpi-users`**
spring.application.name (Eureka)	`{nombre}-service`	→	**`users-service`**
Prefijo externo del Gateway	`/api/{nombre}/**`	→	**`/api/users/**`**

Qué valor determina la ruta

DiscoveryLocatorConfig parte de spring.application.name, normaliza mayúsculas y elimina -service. El nombre del repo no participa del ruteo; solo mantiene trazabilidad entre equipos.

Cada servicio declara su base path por properties

```
app.api.public-path=/api/public/users
app.api.private-path=/api/users


@RequestMapping("${app.api.public-path}")
```

7 Pipeline de seguridad y trazabilidad

Primero se verifica la identidad; después se propaga y se aplican políticas

Spring Security
`SECURITY WEBFILTERCHAIN`
Valida firma, iss, aud, exp y tipo. permitAll solo en rutas documentadas.

Tracing / Correlation
`GlobalFilter · orden 1`
Acepta o crea X-Request-Id y propaga traceparent/tracestate.

LoggingFilter
`GlobalFilter · orden 2`
Registra método, ruta, estado y duración. No guarda bodies ni tokens.

InternalRouteGuard/PublicRouteGuard
`GlobalFilter · orden 3`
Exige serviceId, audience y scope suficientes.

IdentityPropagationFilter
`GlobalFilter · orden 4`
Limpia headers y agrega Principal-Type, User-Id o Service-Id verificados.

RateLimitFilter
`GlobalFilter · orden 5`
Clave por usuario, servicio o IP; estado compartido; rechazo 429 + Retry-After.

Spring Security y los GlobalFilter pertenecen a cadenas distintas: el orden efectivo debe verificarse con pruebas de integración.

8

Cuatro escenarios que debe soportar el Gateway

Cada caso usa el mismo borde, con políticas diferentes

1 · Login público

`POST /api/users/public/auth/login`

permitAll solo para login. users-service autentica y emite el JWT de usuario.

2 · Request de usuario

`GET /api/users/me`

Valida el JWT, propaga identidad y deja la autorización funcional al micro.

3 · Request de servicio

`GET /api/users/profile/{id}`

Micro A vuelve al Gateway con token técnico. No existe llamada directa A ⟶ B.

4 · Destino con falla

`cualquier ruta`

Timeout y circuit breaker devuelven 503 o 504 con Problem Details y traceId.

9

Request autenticado: identidad verificada

El Gateway valida; el microservicio decide si la acción está permitida

1

CLIENTE

Envía el JWT de usuario en Authorization: Bearer.

→

2

SECURITY

Verifica firma, issuer, audience, algoritmo, exp y JWKS.

→

3

GATEWAY

Elimina headers externos y agrega identidad y trazabilidad verificadas.

→

4

MICROSERVICIO

Autoriza rol, pertenencia y ownership; luego ejecuta el caso de uso.

Contrato resultante

```
Authorization: Bearer <user-jwt>

⟶ X-Principal-Type: user
→ X-User-Id / X-User-Roles

→ traceparent + X-Request-Id

401 = token inválido · 403 = autorización funcional rechazada
```

ARQUITECTURA – REQUEST AUTENTICADO: IDENTIDAD VERIFICADA

El Gateway valida; el microservicio decide si la acción está permitida



CLIENTE

Web / Mobile / App

HTTPS

Request

Ejemplo de Request

```
GET /api/pedidos/123 HTTP/1.1
Host: api.miempresa.com
Authorization: Bearer <JWT>
Content-Type: application/json

{ "filtro": "todos" }
```

JWT (ejemplo de payload)

```
{
 "sub": "user-123",
 "iss": "auth.miempresa.com",
 "aud": "mi-api",
 "roles": ["USER"],
 "exp": 1716200000,
 "iat": 1716196400
}
```

SECURITY (en Gateway)

- **Verifica firma del token** — Usa clave pública (JWKS)
- **Valida issuer y audience** — iss == esperado; aud == esperado
- **Valida algoritmo** — Ej: RS256
- **Valida expiración (exp)** — Token vigente
- **Revocación / Blacklist** — Token no revocado

Token válido — Continúa al Gateway

Token inválido — 401 Unauthorized

API GATEWAY

- **Limpieza de headers externos** — Elimina/ignora headers sensibles: X-User-Id, X-Roles, X-Tenant, etc.
- **Enriquecimiento de contexto** — Agrega identidad verificada:
  - userId
  - roles
  - tenantId (si aplica)
  - correlationId / traceId
- **Trazabilidad** — Registra requestId, IP, timestamp, ruta, método
- **Enrutamiento** — Reenvia la solicitud al microservicio objetivo

Headers hacia microservicio (internos y confiables)

- X-User-Id: user-123
- X-Roles: ["USER"]
- X-Tenant-Id: tenant-01
- X-Request-Id: 550e8400-e29b...
- (sin headers externos sensibles)

Infraestructura de Apoyo

- JWKS (cantidad de claves)
- Rate Limiting
- WAF / Protección
- Observability — Logs / Métricas / Trazas / Alertas

MICROSERVICIO

- **Autorización** — Verifica roles y permisos del usuario
- **Pertenencia** — Valida pertenencia a organización / tenant / recurso
- **Ownership / ABAC** — Valida propiedad del recurso o reglas de negocio
- **Caso de uso** — Ejecuta la lógica de negocio
- **Acceso a datos** — Consulta / Actualiza datos

Base de Datos

Respuesta exitosa — 200 OK

No autorizado / Prohibido — 403 Forbidden

Respuesta al cliente

- 200 OK
- 401 / 403 / 500

Leyenda

Solicitud del cliente · Validación de seguridad · Procesamiento en Gateway · Autorización en Microservicio · Respuesta

Principios Clave

- El Gateway autentica (quién es)
- El Microservicio autoriza (qué puede hacer)
- Nunca confiar en headers del cliente
- Identidad y trazabilidad viajan de forma segura y controlada

10

Micro a micro: servicio y usuario no son lo mismo

La llamada vuelve al Gateway y conserva identidad técnica y, si aplica, la delegada

1

MICRO A

Recibe una operación y determina que necesita información del micro B.

2

CREDENCIAL

Obtiene un token técnico con Client Credentials o mTLS y vencimiento corto.

3

GATEWAY

Valida type, serviceId, audience y scope; bloquea tokens de persona.

4

MICRO B

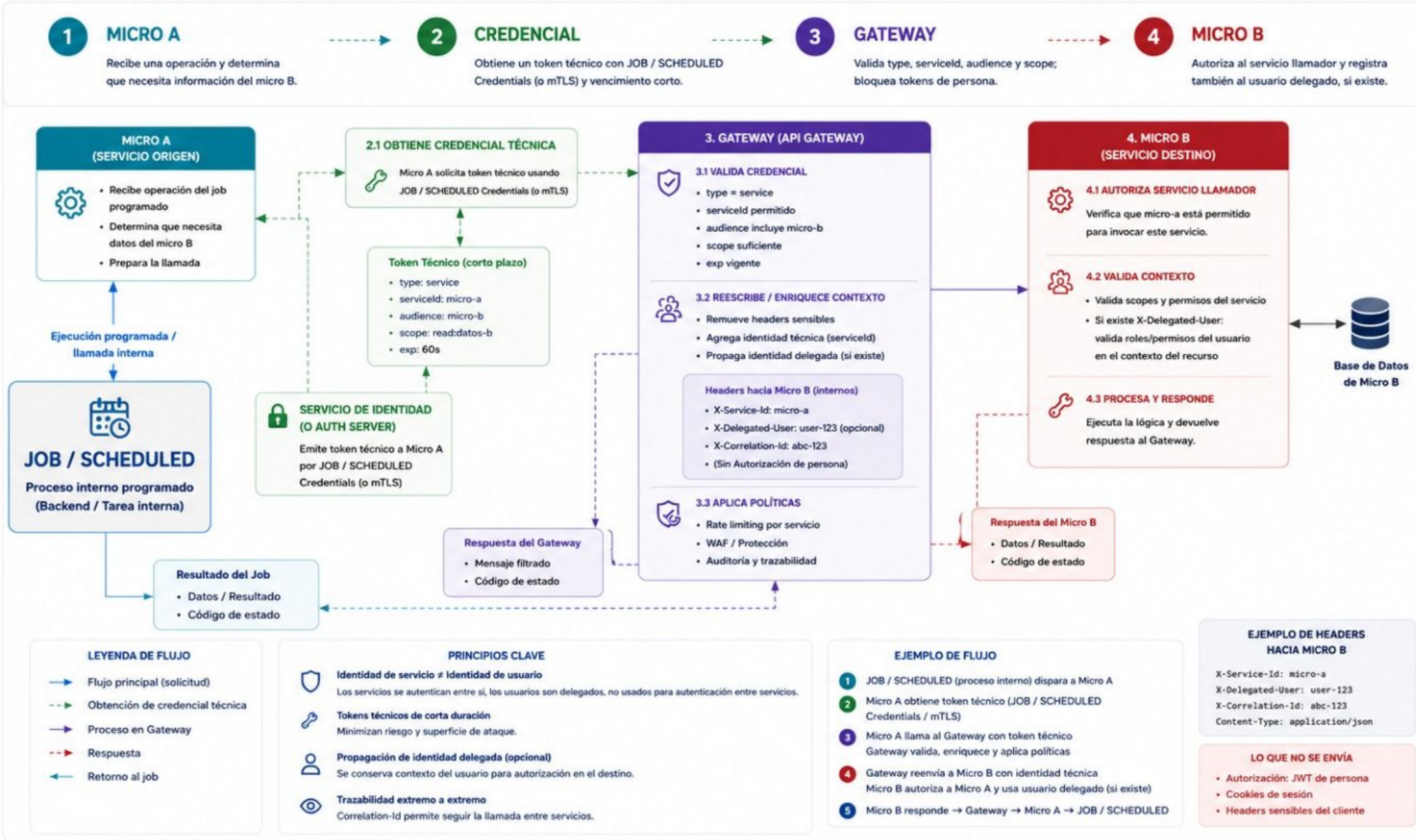
Autoriza al servicio llamador y registra también al usuario delegado, si existe.

Contrato mínimo del token técnico

```
sub=challenges-service · type=service
aud=users-service · scope=users.profile.read
on_behalf_of=<user-id> solo cuando corresponde
Nunca: servicio anónimo ni /auth/token abierto
```

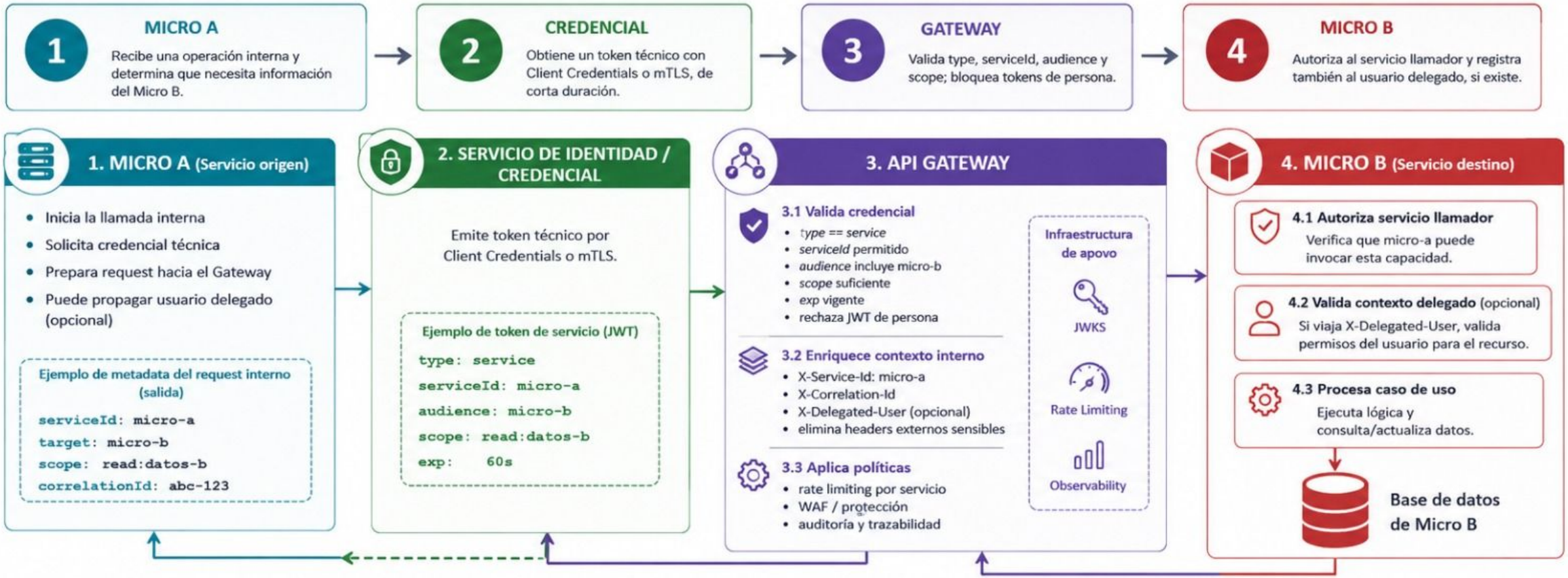
ARQUITECTURA – MICRO A MICRO: SERVICIO Y USUARIO NO SON LO MISMO

La llamada vuelve al Gateway y conserva identidad técnica y, si aplica, la delegada



ARQUITECTURA – MICRO A A MICRO B

Llamada backend-to-backend: identidad de servicio y, si aplica, identidad delegada.



Flujo real

1. Micro A detecta que necesita a Micro B
2. Obtiene token técnico
3. Llama al Gateway
4. Gateway valida y reenvía
5. Micro B procesa y responde

Principios clave

- Identidad de servicio != identidad de usuario
- Entre microservicios se autentica el servicio
- La identidad delegada es opcional y contextual
- Nunca se usa el JWT de persona para autenticar service-to-service

No se envía

- JWT de persona como credencial entre micros
- Cookies de sesión
- Headers sensibles del cliente

11 Componentes del Gateway

Configuración, filtros y servicios de apoyo; no hay entidades de dominio

GatewayApplication

ARRANQUE

@SpringBootApplication

Spring Cloud Gateway · WebFlux

SecurityConfig

VALIDA JWT

+ securityWebFilterChain()

+ ReactiveJwtDecoder ← JWKS

iss · aud · exp · type

DiscoveryLocatorConfig

RUTEO GOBERNADO

include/exclude por serviceId

predicate + path forwarding

InternalRouteGuard

AUTORIZACIÓN TÉCNICA

+ filter(exchange, chain)

lee el principal validado

aud/scope insuficiente → 403
solo /api/servicio/** | entre micros, requiere rol_MS

IdentityPropagationFilter

GLOBALFILTER

+ filter(exchange, chain)

inyecta X-Principal-Type

inyecta User-Id o Service-Id

elimina headers entrantes

ObservabilityFilters

traceparent + requestId

logs sin bodies ni tokens

RateLimitKeyResolver

CONFIGURACIÓN

clave: user / service / IP

+ resolve(exchange)

PrincipalContext

NO VALIDA · NORMALIZA

+ userId(auth)

+ serviceId / scopes(auth)

+ roles / delegatedUser(auth)

ResilienceConfig

RESILIENCE4J + NETTY

timeouts + breaker por serviceId

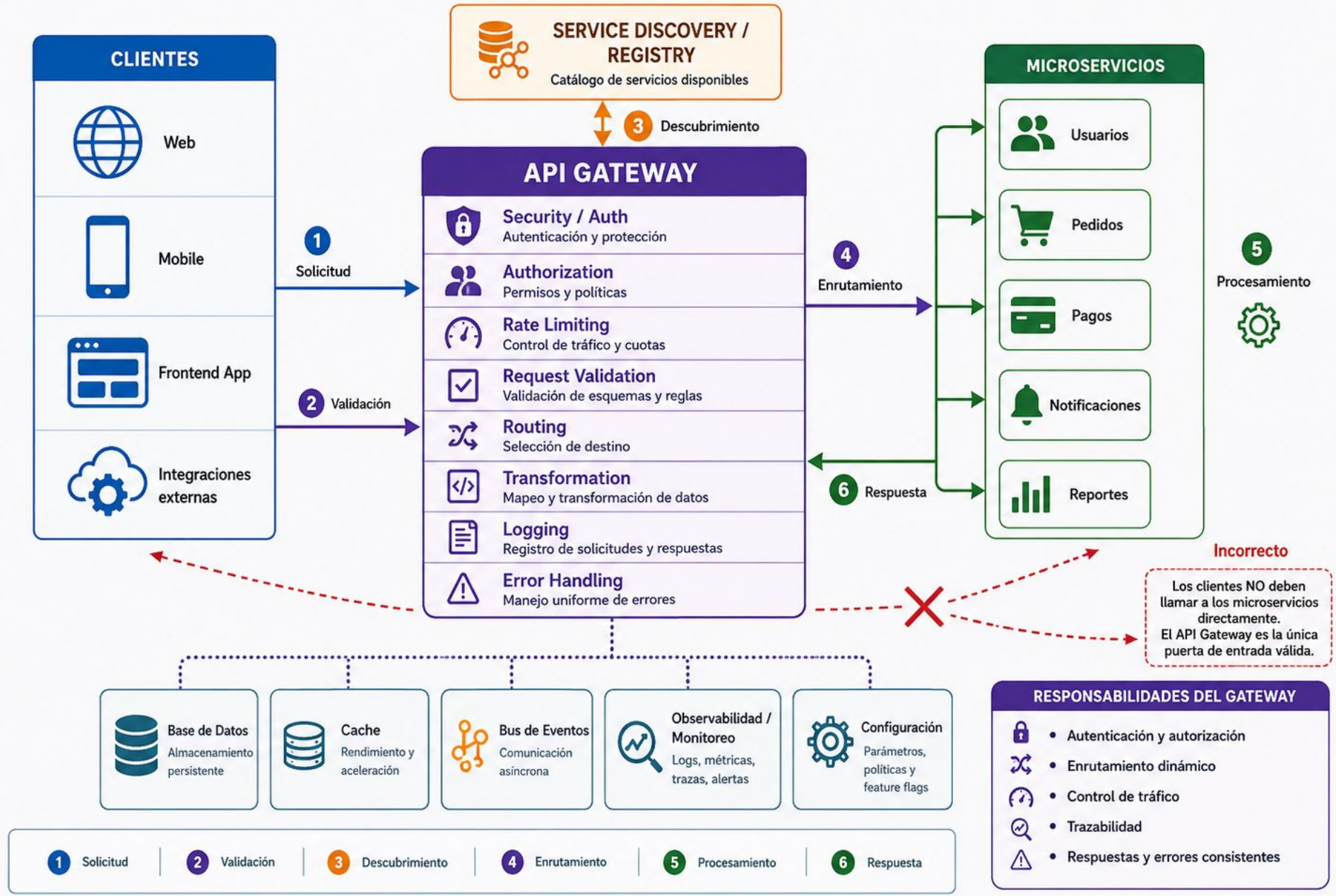
retry idempotente + bulkhead

FallbackController

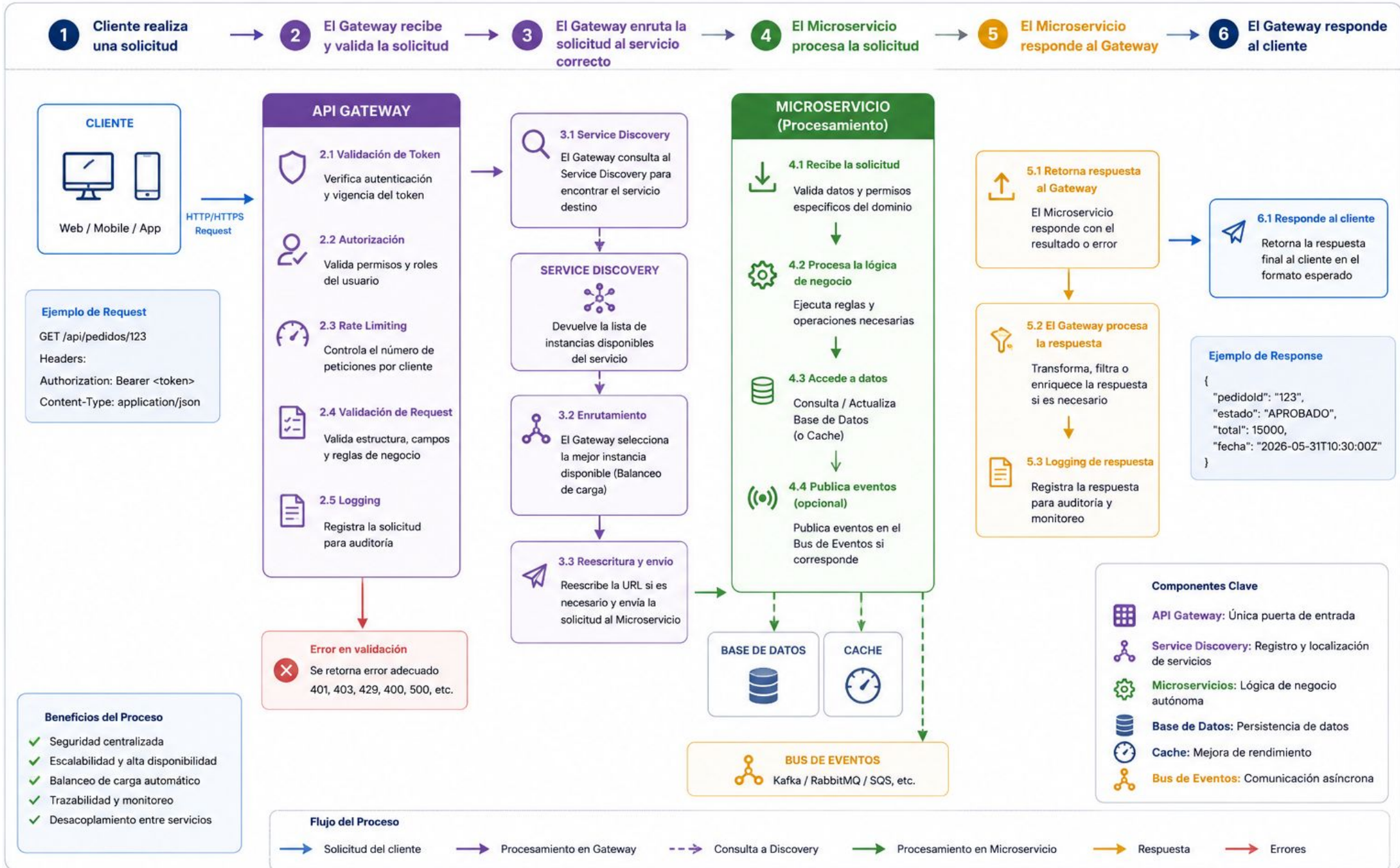
@RequestMapping /fallback → ProblemDetail

ARQUITECTURA GENERAL – API GATEWAY

Puerta única de entrada, seguridad, enrutamiento y observabilidad



PROCESO COMPLETO – API GATEWAY



12 Contratos de identidad y error

Qué espera recibir el Gateway y qué puede confiar el microservicio

LO QUE ESPERA · entrada

```
Authorization: Bearer <jwt>

usuario: sub, roles, type=user
servicio: sub, scope, type=service
siempre: iss, aud, exp, algoritmo permitido
firma RS256 verificable mediante JWKS
```

LO QUE GARANTIZA · salida al micro

```
X-Principal-Type: user | service
X-User-Id o X-Service-Id
X-User-Roles o X-Service-Scopes
```

traceparent + X-Request-Id

```
Confiable solo desde la red privada del Gateway.
```

Respuestas emitidas por el propio Gateway

401	**Token requerido ausente, inválido o vencido**	Reautenticar; permitAll no exige token
403	**Ruta interna sin audience o scope suficiente**	El micro emite otros 403 por reglas de negocio
429	**Límite de la ruta superado**	Respetar Retry-After; no reintentar a ciegas
503	**Sin instancia o circuit breaker abierto; timeout → 504**	Body Problem Details con traceId

13

Endpoints operativos del Gateway

Deben existir para operar la plataforma, pero no quedar expuestos a Internet

El Gateway responde pocas rutas propias; el resto se reenvía a los microservicios:

`GET /actuator/health/liveness | readiness`

El orquestador o Compose comprueba proceso vivo y capacidad de recibir tráfico.

`GET /actuator/prometheus`

Métricas internas: latencia, throughput, 4xx/5xx, retries y estado de breakers.

`forward:/fallback/**`

Destino interno del circuit breaker. Atiende todos los métodos y devuelve Problem Details.

Actuator usa puerto o red de management. /actuator/gateway, si se habilita, queda read-only e interno.

14

Service Discovery con Eureka

El Gateway resuelve instancias sin guardar direcciones fijas

El problema que resuelve

SIN discovery

El Gateway tendría que guardar host y puerto de cada instancia. Un reinicio o escalado volvería obsoleta esa configuración.

CON Eureka

El Gateway descarga el registro, mantiene una copia local y el LoadBalancer elige una instancia disponible.

Los 3 elementos que publica o renueva cada microservicio

1

Nombre lógico

`spring.application.name`

Identifica al servicio. Convención: users-service.

2

Dirección y puerto

`host + port`

Ubicación actual de cada instancia; nunca se escribe en las rutas externas.

3

Lease y estado

`heartbeat 30 s · status UP`

El heartbeat renueva el lease. Actuator solo influye si se habilita la integración de health checks.

15

El ciclo real de Eureka

Registro, lease, caché y balanceo: no hay una consulta por cada request

1

REGISTRO

La instancia publica serviceId, host, puerto y metadata.

2

LEASE

Heartbeat cada 30 s; la expiración habitual es 90 s y no es inmediata.

3

CACHÉ

El Gateway actualiza periódicamente su copia del registro de Eureka.

4

BALANCEO

lb:// selecciona una instancia UP; timeouts cubren información todavía obsoleta.

Qué significa realmente lb://

```
uri: lb://USERS-SERVICE

El LoadBalancer consulta la copia local obtenida de Eureka y elige una instancia.
Self-preservation o cachés pueden retrasar una baja: por eso discovery no reemplaza timeout ni circuit breaker.
```

ARQUITECTURA – EL CICLO REAL DE EUREKA

Registro, lease, caché y balanceo: no hay una consulta por cada request

1 REGISTRO
La instancia publica serviceId, host, puerto y metadata.

2 LEASE
Heartbeat cada 30 s; la expiración habitual es 90 s y no es inmediata.

3 CACHÉ
El Gateway actualiza periódicamente su copia del registro de Eureka.

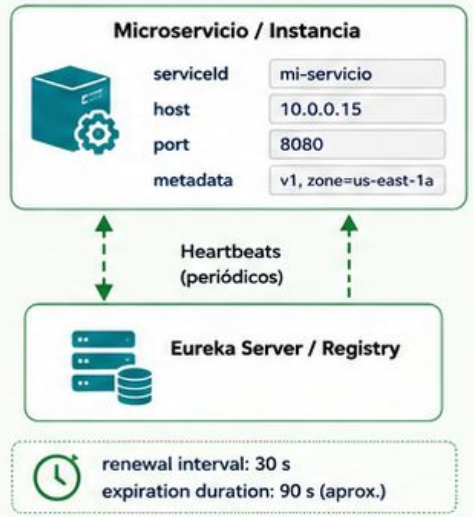
4 BALANCEO
lb:// selecciona una instancia IP; timeouts cubren información todavía obsoleta.

1. REGISTRO



- Alta inicial de la instancia
- Publica su endpoint y metadatos
- Queda visible para descubrimiento

2. LEASE / HEARTBEAT



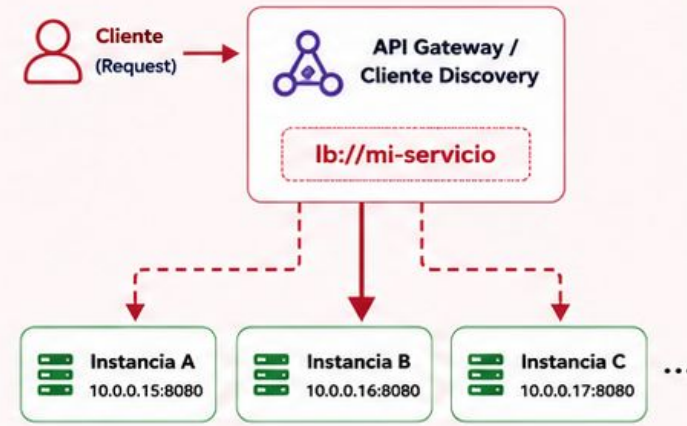
- La instancia renueva su lease
- Si deja de latir no desaparece instantáneamente
- La baja efectiva tiene ventana de tolerancia

3. CACHÉ LOCAL



- No consulta Eureka en cada request
- Sincroniza periódicamente el registro
- Puede existir desfase temporal

4. BALANCEO



- Elige una instancia desde la caché local
- Balancea entre instancias disponibles
- Si una IP ya no responde, actúan retries/timeouts/circuit breaker

EL SECUENCIA REAL (FLUJO DE PUNTA A PUNTA)

1. La instancia se registra en Eureka
2. Renueva lease con heartbeats
3. El Gateway actualiza su caché del registro
4. Llega una request del cliente
5. El Gateway resuelve lb:// usando su caché
6. Envía la llamada a una instancia
7. Si la instancia está caída pero aún en caché, puede fallar y reintentar

INFRAESTRUCTURA DE APOYO

Timeouts | Retries | Circuit Breaker | Observabilidad (Logs, Métricas, Trazas)

PRINCIPIOS CLAVE

- Eureka es registro y descubrimiento, no routing por request
- La información puede quedar momentáneamente desactualizada
- La resiliencia compensa la naturaleza eventual del registro
- El balanceo ocurre del lado del cliente/gateway usando caché

LEYENDA (TIPOS DE FLUJO)

Registro (Alta inicial) | Heartbeat (Lease) | Sync de caché (Pull) | Request / Balanceo | Respuesta / Error

16 Configuración mínima de Eureka

Cada componente registra y consulta según su responsabilidad

Convención de nombres · kebab-case + sufijo -service

`users-service` · `mailing-service` · `api-gateway`

```
spring.application.name=users-service
eureka.client.service-url.defaultZone=${EUREKA_URL:http://localhost:8761/eureka/}
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=false
eureka.client.healthcheck.enabled=true
```

- **`spring.application.name`** — Valor canónico en minúsculas. Eureka puede mostrarlo en mayúsculas; el locator lo normaliza.
- **`eureka.client.service-url.defaultZone`** — Dirección del servidor Eureka, siempre configurable por ambiente.
- **`register-with-eureka`** — true en los micros. En el Gateway es opcional; si se registra, debe excluirse del locator.

fetch-registry

true en el Gateway. Los micros pueden usar false si llaman al Gateway por DNS interno; si lo descubren por Eureka, necesitan true.

17

Ruteo dinámico gobernado

Descubrimiento automático no significa exposición automática

application.yml + DiscoveryLocatorConfig.java

```
discovery.locator:
  enabled: true
  lower-case-service-id: true
```

include-expression: <solo servicios admitidos>

```
Regla:
USERS-SERVICE → users
predicate: /api/users/**
```

uri: lb://USERS-SERVICE

```
forwarded path: /api/users/**
```

Condiciones del diseño

Exposición controlada

Allowlist o regla de inclusión; Gateway, Eureka, Actuator e infraestructura quedan fuera.

Contrato único de nombres

El serviceId deriva la URL externa y se valida en CI.

Políticas por servicio

Breaker, timeout y rate limit reciben serviceId como clave.

Sin autoruta

api-gateway nunca genera una ruta hacia sí mismo.

Agregar un micro = nombre válido + opt-in + contrato OpenAPI + pruebas. No alcanza con registrarlo en Eureka.

Especificación pendiente de verificación

Fijar versiones de Spring Boot/Cloud, implementar predicate y path forwarding, probar con 2 servicios y confirmar que servicios desconocidos no se exponen.

18

Una ruta completa, sin ambigüedad

Debe quedar definido qué se compara y qué path recibe el backend

1

SERVICE ID

USERS-SERVICE se normaliza y se transforma en users.

2

PREDICATE

El request entrante coincide con /api/users/**.

3

LOAD BALANCER

lb://USERS-SERVICE elige una instancia desde la caché de Eureka.

4

PATH ENVIADO

Nuestro contrato conserva /api/users/me hasta el controller.

Contrato elegido

```
Entrada: GET /api/users/me

Destino: lb://USERS-SERVICE
Path enviado: /api/users/me

Controller: @RequestMapping("${app.api.public-path}")

Si se usa RewritePath, documentar y probar el path final; no mezclar alternativas.
```

ARQUITECTURA – UNA RUTA COMPLETA, SIN AMBIGÜEDAD

Debe quedar definido qué se compara y qué path recibe el backend

CLIENTE

Navegador / App / API Client

HTTP REQUEST

GET `/api/users/me`

1 SERVICE ID

Identificador lógico del servicio destino

USERS-SERVICE
(serviceId lógico)

NORMALIZACIÓN

users
(nombre normalizado)

Qué significa:

- Identifica el servicio destino
- No se compara contra la URL
- Se normaliza para discovery
- Usado por lb:// y discovery

Configuración del servicio y normalización

2 PREDICATE

La URL entrante se compara contra el predicate de la ruta

Ejemplo de Ruta (config):

```
id: users-route
uri: lb://USERS-SERVICE
predicates: Path=/api/users/**
```

Request entrante:

`/api/users/me`

MATCH
Coincide con Path=/api/users/**

Qué significa:

- El request entrante se compara con el Path predicate
- Si coincide, la ruta se activa

3 LOAD BALANCER

El API Gateway / Load Balancer usa caché local de discovery

API GATEWAY / LOAD BALANCER

CACHÉ LOCAL DEL REGISTRO
(Eureka / Discovery)

Instancias disponibles de users

Instancia A	10.0.0.11:8080
Instancia B	10.0.0.12:8080
Instancia C	10.0.0.13:8080
...	

Elige 1 instancia disponible

Qué significa:

- Elige una instancia disponible
- Usa caché local del registro
- Aplica balanceo entre instancias
- No consulta Eureka en cada request

4 PATH ENVIADO

El request se reenvía a la instancia seleccionada sin cambiar el path

Instancia seleccionada
10.0.0.12:8080

Controller
UsersController

Path recibido por el backend:
`/api/users/me`

Qué significa:

- El contrato conserva el path
- El backend recibe /api/users/me
- Luego entra al controller correspondiente
- No se modifica el path en tránsito

FLUJO END-TO-END (6 PASOS)

1. Cliente envía /api/users/me
2. Gateway evalúa la ruta
3. Path predicate hace match
4. lb:// resuelve una instancia
5. Se reenvía el request
6. El backend recibe el mismo path

EJEMPLO DE RUTA

```
id: users-route
uri: lb://USERS-SERVICE
predicates: Path=/api/users/**
```

PRINCIPIOS CLAVE

- serviceId ≠ path
- El predicate compara la URL entrante
- lb:// resuelve instancia, no cambia el contrato
- El backend recibe el path esperado

ERROR COMÚN

Confundir el serviceId con el path

LEYENDA (TIPOS DE FLUJO)

Solicitud (entrante) · Evaluación de ruta (gateway) · Discovery / Balanceo (lb://) · Reenvío (a instancia) · Respuesta (de regreso)

19

Resiliencia y observabilidad

El circuit breaker es una capa; la protección completa necesita límites

Timeout + bulkhead

Connect/response timeout por ruta, pool limitado y tope de concurrencia para que un destino lento no agote el Gateway.

Circuit breaker

Uno por serviceId. Abre solo el destino con fallas y devuelve 503 mediante fallback.

Retry seguro

Solo GET o solicitudes con idempotency key; backoff + jitter. Un 429 respeta Retry-After.

Trazabilidad y alertas

traceparent, requestId, p95/p99, tasa de error, retries, rechazos 429 y estado de cada breaker.

Para IA: rate limit → bulkhead → timeout → retry controlado → circuit breaker → fallback. El breaker solo no protege conexiones ni memoria.

20

Pruebas obligatorias antes de integrar

La arquitectura queda aprobada cuando pasan estas cuatro familias

1 · Ruteo

`2 servicios reales`

Valida derivación, path final, exclusiones, servicio desconocido 404 y ausencia de autoruta.

2 · Seguridad

`JWT y headers`

401 inválido; 403 interno; aud/scope; headers falsificados eliminados y reemplazados.

3 · Discovery

`Eureka y balanceo`

Dos instancias, baja eventual, caché durante caída de Eureka y ningún acceso directo.

4 · Resiliencia

`fallas controladas`

Timeout 504, breaker aislado, retry idempotente, 429 + Retry-After y traza end-to-end.

En resumen

1	**Puerta única verificable**	La red bloquea accesos directos; solo el Gateway está expuesto.
2	**Dos niveles de autorización**	El Gateway protege la frontera técnica; cada micro decide el negocio.
3	**Ruteo dinámico gobernado**	Solo servicios admitidos; sin autoruta ni exposición automática.
4	**Identidad completa**	Distingue usuario, servicio y contexto delegado cuando corresponde.
5	**Eureka resuelve instancias**	Usa registro y caché; health checks y bajas requieren configuración.
6	**Fallas aisladas y probadas**	Timeout, bulkhead, retry seguro, breaker por destino y observabilidad.

Definition of Done: implementación ejecutable + pruebas con 2 servicios + contratos versionados